

МЕТОДИЧЕСКИЕ УКАЗАНИЯ К КУРСОВОЙ РАБОТЕ (ПРОЕКТУ)

по дисциплине «Методы и технологии программирования»

1 ЦЕЛЬ И ЗАДАЧИ КУРСОВОЙ РАБОТЫ (ПРОЕКТА)

Цель курсовой работы (проекта) — продемонстрировать способность создавать сложный программный продукт с использованием современных языков программирования (Python, C++, Go, Rust), технологий и архитектурных подходов, пройдя полный цикл разработки: от идеи до развёртывания

Задачи курсового проекта:

1. Провести анализ предметной области и сформулировать требования к программному продукту
2. Спроектировать архитектуру приложения с использованием современных методологий
3. Реализовать программный продукт с применением изученных технологий
4. Обеспечить качество кода через тестирование и статический анализ
5. Подготовить документацию и провести презентацию результатов

2 ТРЕБОВАНИЯ К КУРСОВОЙ РАБОТЕ (ПРОЕКТУ)

2.1 Технологический стек (на выбор в зависимости от варианта)

Язык	Назначение	Ключевые технологии
Python	Бэкенд, аналитика, ИИ, оркестрация	FastAPI, Django, Polars, LangChain, Pydantic V2
Go	Сетевые сервисы, микросервисы, агенты	Gin, NATS, gRPC, горутины, каналы
Rust	Высокопроизводительные компоненты, безопасность	PyO3, Actix-web, Tokio, Serde
C++	Системное программирование, графика, вычисления	C++20/23, Qt, OpenGL, pybind11

2.2 Обязательные компоненты проекта

1. **Система контроля версий** — Git, семантические коммиты, Git flow
2. **Тестирование** — модульные тесты (pytest, Go test, cargo test)
3. **Контейнеризация** — Docker, Docker Compose]
4. **Документирование** — README, API документация, диаграммы
5. **Безопасность** — SAST-анализ, проверка зависимостей

6. Интеграция ИИ-инструментов — использование AI-ассистентов в разработке

2.3 Структура пояснительной записки (обязательная)

Введение — актуальность, цель, задачи

1 Аналитическая часть — обзор предметной области, анализ аналогов

2 Проектная часть — архитектура, диаграммы (UML, PlantUML, Mermaid)

3 Технологическая часть — обоснование выбора языка и технологий

4 Реализация — описание ключевых компонентов, фрагменты кода

5 Тестирование — результаты тестов, анализ производительности

Заключение — выводы, перспективы развития

Список использованной литературы

Приложения — код, скриншоты

3 ИНДИВИДУАЛЬНЫЕ ВАРИАНТЫ КУРСОВОГО ПРОЕКТА

Категория : Обеспечение информационной безопасности систем

Вариант 1. Система двухфакторной аутентификации (2FA) с поддержкой TOTP и push-уведомлений

- **Предметная область:** Аутентификация и управление доступом
- **Технологии:** Python + FastAPI + Redis + Firebase Cloud Messaging
- **Описание:** Сервис для обеспечения двухфакторной аутентификации в веб-приложениях. Генерация TOTP-кодов (RFC 6238), отправка push-уведомлений на мобильное приложение. API для интеграции с существующими системами. Админ-панель для управления устройствами пользователей.

Вариант 2. Инструмент статического анализа кода (SAST) для поиска уязвимостей типа injection

- **Предметная область:** Безопасная разработка
- **Технологии:** Python + tree-sitter + Rust
- **Описание:** Создание статического анализатора для обнаружения SQL-инъекций, XSS и Command Injection в коде на Python и Go. Использование библиотеки tree-sitter для построения AST, анализ потоков данных, выявление небезопасных вызовов. Интеграция с CI/CD (GitHub Actions) и генерация отчётов.

Вариант 3. Система обнаружения вторжений (IDS) на основе машинного обучения

- **Предметная область:** Сетевая безопасность
- **Технологии:** Python + scikit-learn + Go + PCAP
- **Описание:** Разработка модуля анализа сетевого трафика в реальном времени. Захват пакетов (Go), извлечение признаков, классификация аномалий с помощью ML-моделей (Random Forest, нейросети). Визуализация атак, алерты в Telegram/Slack.

Вариант 4. Платформа управления уязвимостями (Vulnerability Management)

- **Предметная область:** Управление рисками
- **Технологии:** Python + FastAPI + PostgreSQL + NVD API
- **Описание:** Система для сбора и анализа данных об уязвимостях из открытых источников (CVE, NVD). Сопоставление с инвентаризацией ПО организации, приоритизация на основе CVSS и контекста. Веб-интерфейс для отслеживания статуса исправления.

Вариант 5. Реализация протокола безопасного обмена сообщениями с end-to-end шифрованием

- **Предметная область:** Криптография, защита коммуникаций
- **Технологии:** Rust + Python + Signal Protocol (библиотеки)
- **Описание:** Разработка защищённого мессенджера с использованием протокола двойного храповика (Double Ratchet). Rust-ядро для криптографических операций, Python-клиент с графическим интерфейсом (Tkinter/PyQt). Поддержка групповых чатов, PFS (Perfect Forward Secrecy).

Вариант 6. Инструмент для анализа конфигураций Docker-образов на соответствие CIS Benchmarks

- **Предметная область:** Безопасность контейнеров
- **Технологии:** Go + Python + Docker API
- **Описание:** Сканер Docker-образов и запущенных контейнеров для проверки соответствия стандартам CIS Docker Benchmark. Выявление небезопасных настроек (привилегированные режимы, открытые порты, уязвимые версии). Генерация отчётов с рекомендациями по исправлению [ЛР №15].

Вариант 7. Система моделирования и анализа деревьев атак (графовый метод)

- **Предметная область:** Анализ защищённости, моделирование угроз
- **Технологии:** Python + NetworkX + Go (визуализация)
- **Описание:** Разработка инструмента для построения и анализа деревьев атак на информационную систему. Графовое представление векторов

атак, расчёт метрик риска (вероятность успеха, стоимость реализации). Автоматическая генерация отчётов с рекомендациями по митигации. Визуализация графов в интерактивном веб-интерфейсе.

Вариант 8. Программа для аудита безопасности веб-приложений

- **Предметная область:** Тестирование на проникновение
- **Технологии:** Python + requests + BeautifulSoup + Playwright
- **Описание:** Инструмент для автоматического сканирования веб-приложений на наличие типовых уязвимостей: XSS, SQLi, CSRF, небезопасные заголовки. Возможность краулинга сайта, анализа форм, генерации Proof-of-Concept. Отчёт в формате HTML/PDF.

Вариант 9. Система обнаружения аномалий в поведении пользователей (UEBA)

- **Предметная область:** Мониторинг инцидентов
- **Технологии:** Python + pandas + scikit-learn + Elasticsearch
- **Описание:** Анализ логов событий (входы, действия, доступ к данным) для выявления подозрительной активности. Построение профилей нормального поведения, выявление отклонений (необычное время, местоположение, объёмы данных). Визуализация в Kibana, алерты при критических аномалиях.

Вариант 10. Инструмент для анализа цепочки поставок (supply chain security)

- **Предметная область:** Безопасность разработки
- **Технологии:** Python + Go + OSS Index API + SPDX
- **Описание:** Сервис для проверки зависимостей проекта на наличие известных уязвимостей (CVE), лицензионной совместимости и устаревших версий. Поддержка экосистем PyPI, npm, Go modules, Cargo. Генерация SBOM (Software Bill of Materials) в формате SPDX. Интеграция с системами управления уязвимостями [ЛР №15].

Категория: Корпоративные системы и автоматизация

Вариант 11. Система документооборота с ИИ-классификацией

- **Предметная область:** Делопроизводство, управление документами
- **Технологии:** Python + Go
- **Описание:** Разработка системы для автоматической обработки входящих документов. Python-оркестратор управляет очередью, Go-агенты выполняют OCR (оптическое распознавание) и классификацию документов по типам с использованием ML-моделей. Результат — автоматическое распределение по исполнителям

Вариант 12. Интеграционная шина данных для 1С и внешних систем

- **Предметная область:** Интеграция корпоративных систем
- **Технологии:** Go + Python
- **Описание:** Разработка сервиса обмена данными между 1С:Предприятие и внешними системами через HTTP-сервисы с поддержкой версионирования и логированием. Механизм трансформации данных в различных форматах (JSON, XML, SOAP)

Вариант 13. Подсистема учёта аудиторного фонда университета

- **Предметная область:** Образование, управление ресурсами
- **Технологии:** Python + C++
- **Описание:** Разработка модуля для 1С:Университет с возможностью интеграции с системой создания расписания. Визуализация занятости аудиторий, оптимизация распределения, 3D-карта кампуса с отображением свободных помещений

Вариант 14. Система управления проектами с аналитикой

- **Предметная область:** Управление проектами
- **Технологии:** FastAPI + React + PostgreSQL
- **Описание:** Аналог Trello/Jira с расширенной аналитикой. Диаграммы Ганта, расчёт метрик (время выполнения, загрузка ресурсов), прогнозирование сроков на основе исторических данных. WebSocket для обновлений в реальном времени [ЛР №10].

Вариант 15. Платформа для А/В тестирования веб-интерфейсов

- **Предметная область:** Маркетинг, веб-аналитика
- **Технологии:** Go + Python
- **Описание:** Go-сервис распределения трафика между версиями страниц, Python-сервис для статистического анализа результатов (t-тест, метод Монте-Карло). Дашборд с результатами экспериментов в реальном времени.

Вариант 16. Система автоматизации деятельности управляющей компании ЖКХ

- **Предметная область:** Жилищно-коммунальное хозяйство
- **Технологии:** C# + Python
- **Описание:** Разработка подсистем: аутентификация (OpenID Connect), автоматический сбор показаний приборов учёта (LoRaWAN, MQTT), уведомление жителей (email, SMS, push). Веб-интерфейс для диспетчеров

Вариант 17. Модуль автоматической нормализации справочников

- **Предметная область:** Управление нормативно-справочной информацией (НСИ)
- **Технологии:** Python + ML
- **Описание:** Разработка модуля с набором алгоритмов и моделей машинного обучения для нормализации записей справочников. Очистка данных, устранение дубликатов, приведение к единому формату. Актуально для справочников ФСТЭК, ОКВЭД

Вариант 18. Рекомендательная система недвижимости

- **Предметная область:** Недвижимость
- **Технологии:** Python + FastAPI + React
- **Описание:** Сервис подбора недвижимости на основе предпочтений пользователя. Коллаборативная фильтрация, анализ истории просмотров, геопространственный поиск. Интеграция с картами (Yandex Maps API)

Категория: Аналитика и Data Science

Вариант 19. Платформа для streaming-аналитики в реальном времени

- **Предметная область:** Большие данные, аналитика
- **Технологии:** Rust + Python + Apache Arrow
- **Описание:** Rust-компонент для высокопроизводительной обработки потоков данных (оконные агрегации, фильтрация), Python-компонент для визуализации и управления. Использование Apache Arrow для эффективного обмена данными без копирования [ЛР №14].

Вариант 20. Система мониторинга и прогнозирования сбоев серверов

- **Предметная область:** DevOps, мониторинг
- **Технологии:** Go + Python + Prometheus
- **Описание:** Сбор метрик производительности (CPU, память, диски) с серверов виртуализации. ML-модели для прогнозирования сбоев на основе анализа временных рядов. Система раннего предупреждения с интеграцией в Telegram/Slack

Вариант 21. Интеллектуальный парсинг вакансий с hh.ru

- **Предметная область:** Рекрутмент, анализ рынка труда
- **Технологии:** Go + Python + NLP
- **Описание:** Go-сборщик для параллельного парсинга тысяч вакансий, Python-аналитик для извлечения навыков, классификации уровней

(junior/middle/senior), построения графиков востребованности технологий по регионам [ЛР №14].

Вариант 22. Генетический алгоритм оптимизации маршрутов доставки

- **Предметная область:** Логистика
- **Технологии:** C++ + Python + OpenCL
- **Описание:** C++-ядро с GPU-ускорением (OpenCL) для решения задачи коммивояжёра на больших графах. Python-интерфейс для визуализации маршрутов на карте, учёт ограничений (время доставки, грузоподъёмность).

Вариант 23. Система анализа тональности отзывов с визуализацией

- **Предметная область:** Маркетинг, аналитика
- **Технологии:** Python + Transformers + FastAPI
- **Описание:** Сервис для сбора отзывов (маркетплейсы, соцсети), анализа тональности (ruBERT), выделения ключевых тем (тематическое моделирование). Дашборд с динамикой изменения отношения к продукту.

Вариант 24. Прогнозирование временных рядов для финансовых рынков

- **Предметная область:** Финансы, трейдинг
- **Технологии:** Rust + Python + WebSocket
- **Описание:** Rust-библиотека для быстрого расчёта технических индикаторов (SMA, RSI, MACD). Python-сервис для получения котировок в реальном времени (WebSocket) и применения ML-моделей (LSTM) для прогнозирования.

Вариант 25. Сервис классификации медицинских изображений

- **Предметная область:** Медицина
- **Технологии:** Python + PyTorch + FastAPI
- **Описание:** Веб-сервис для загрузки медицинских снимков (рентген, КТ) и классификации патологий с помощью свёрточной нейросети. Объяснимость решений — визуализация областей, на которые обратила внимание модель (Grad-CAM)

Вариант 26. Определение дубликатов вопросов на StackOverflow

- **Предметная область:** Обработка естественного языка
- **Технологии:** Python + Transformers + Elasticsearch
- **Описание:** Система поиска похожих вопросов по тексту. Использование sentence-transformers для получения эмбедингов,

Elasticsearch для быстрого поиска по сходству. Веб-интерфейс для тестирования

Категория: Высоконагруженные системы и микросервисы

Вариант 27. API-шлюз с rate limiting и аутентификацией

- **Предметная область:** Инфраструктура веб-сервисов
- **Технологии:** Go
- **Описание:** Разработка высокопроизводительного API-шлюза на Go. Поддержка JWT-аутентификации, rate limiting (token bucket, sliding window), маршрутизация запросов к бэкенд-сервисам, сбор метрик [ЛР №15].

Вариант 28. Распределённая система кэширования

- **Предметная область:** Базы данных, кэширование
- **Технологии:** Rust + Tokio
- **Описание:** Реализация in-memory key-value хранилища с поддержкой кластеризации (consistent hashing), репликации и персистентности. Асинхронная обработка запросов (Tokio), протокол, совместимый с Redis.

Вариант 29. Сервис сокращения ссылок с аналитикой

- **Предметная область:** Веб-сервисы
- **Технологии:** Go + PostgreSQL + Redis
- **Описание:** Аналог bit.ly. Go-бэкенд с генерацией коротких кодов, редиректами, сбором статистики (переходы, гео, рефереры). Redis для кэширования популярных ссылок. Дашборд для владельцев ссылок [ЛР №10].

Вариант 30. Чат-система с WebSocket и комнатами

- **Предметная область:** Коммуникации
- **Технологии:** Go + WebSocket + React
- **Описание:** Высоконагруженный чат-сервер на Go (горутины на каждое соединение). Поддержка комнат, приватных сообщений, истории, push-уведомлений. Веб-клиент на React. Тестирование под нагрузкой до 10K одновременных соединений [ЛР №13].

Вариант 31. Система бронирования столиков в ресторанах

- **Предметная область:** HoReCa
- **Технологии:** Python + FastAPI + PostgreSQL

- **Описание:** Бэкенд для мобильного приложения бронирования. Управление доступностью, карта зала с визуализацией свободных мест, интеграция с картами для навигации, уведомления о подтверждении

Вариант 32. Маркетплейс услуг с эскароу-системой

- **Предметная область:** Электронная коммерция
- **Технологии:** Python + Go + Blockchain (симуляция)
- **Описание:** Платформа для заказа услуг с безопасными расчётами. Go-микросервис для обработки платежей и эскароу-счетов, Python-бэкенд для управления заказами и пользователями. Симуляция блокчейн-логики для прозрачности.

Вариант 33. Платформа для онлайн-курсов с видео-стримингом

- **Предметная область:** Образование
- **Технологии:** Go + Python + Nginx
- **Описание:** Go-сервис для стриминга видео с адаптивным битрейтом (HLS), Python-бэкенд (Django) для управления курсами, пользователями, тестами. Интеграция с CDN, защита контента [ЛР №11].

Вариант 34. Система логирования и мониторинга микросервисов

- **Предметная область:** DevOps, наблюдаемость
- **Технологии:** Rust + Go + ELK
- **Описание:** Rust-агент для сбора логов с минимальным потреблением ресурсов, Go-агрегатор для обработки и записи в Elasticsearch. Веб-интерфейс для поиска и визуализации, алерты при аномалиях [ЛР №11].

Категория: Системное программирование и производительность

Вариант 35. Высокопроизводительная библиотека для линейной алгебры

- **Предметная область:** Научные вычисления
- **Технологии:** C++20 + Python + pybind11
- **Описание:** Реализация базовых операций линейной алгебры (умножение матриц, обращение, разложение) с использованием SIMD-инструкций (AVX2/AVX-512). Python-биндинги для использования из NumPy-подобного API. Сравнение производительности с OpenBLAS [ЛР №9].

Вариант 36. Эмулятор CHIP-8 на Rust с WebAssembly

- **Предметная область:** Эмуляция, ретрогейминг
- **Технологии:** Rust + WebAssembly + JavaScript
- **Описание:** Эмулятор классического CHIP-8, скомпилированный в WebAssembly для запуска в браузере. Поддержка ROM-файлов, отображение графики, эмуляция звука. Веб-интерфейс с кнопками управления

Вариант 37. Собственная in-memory база данных "ключ-значение"

- **Предметная область:** Базы данных
- **Технологии:** C++17 + Boost + gRPC
- **Описание:** Реализация ACID-совместимой in-memory БД с поддержкой транзакций, персистентностью (snapshot + WAL), сетевым доступом через gRPC. Поддержка простых запросов и TTL

Вариант 38. HTTP/1.1 сервер с нуля на C

- **Предметная область:** Сетевое программирование
- **Технологии:** C + sockets + epoll
- **Описание:** Реализация полноценного HTTP/1.1 сервера с поддержкой keep-alive, виртуальных хостов, статических файлов. Неблокирующий ввод-вывод (epoll). Сравнение производительности с nginx

Вариант 39. Компилятор для учебного языка программирования

- **Предметная область:** Теория компиляции
- **Технологии:** C++ + LLVM + Flex/Bison
- **Описание:** Разработка компилятора для простого императивного языка. Лексический и синтаксический анализ (Flex/Bison), генерация кода через LLVM, поддержка базовых типов, функций, условных операторов и циклов

Вариант 40. Парсер JSON с нуля на Rust

- **Предметная область:** Системное программирование
- **Технологии:** Rust + Serde-like API
- **Описание:** Реализация высокопроизводительного парсера JSON с поддержкой полной спецификации. Безопасная работа с памятью (panics), zero-copy parsing для строк. API, совместимое с Serde

Вариант 41. Распределённая система обмена сообщениями (Message Broker)

- **Предметная область:** Интеграция систем
- **Технологии:** Go + NATS + gRPC

- **Описание:** Упрощённый аналог RabbitMQ/NATS. Поддержка очередей, публикации/подписки, гарантий доставки (at-least-once). Go-клиенты на gRPC, веб-интерфейс для мониторинга [ЛР №13].

Вариант 42. Трассировщик лучей в реальном времени

- **Предметная область:** Компьютерная графика
- **Технологии:** C++ + Vulkan/Optix + CUDA
- **Описание:** Программа для рендеринга трёхмерных сцен с использованием аппаратного трассировки лучей (RTX). Поддержка материалов, освещения, теней. Интерактивное изменение камеры и параметров сцены

Категория: Мобильные и десктопные приложения

Вариант 43. Приложение для управления личными финансами

- **Предметная область:** Персональные финансы
- **Технологии:** Python + Kivy + SQLite
- **Описание:** Кроссплатформенное мобильное приложение для учёта доходов/расходов. Категоризация транзакций, визуализация (диаграммы), финансовые цели, напоминания о платежах. Защита данных (шифрование локальной БД)

Вариант 44. Фитнес-трекер с интеграцией датчиков устройства

- **Предметная область:** Здоровье, спорт
- **Технологии:** Python + Kivy + Health API
- **Описание:** Мобильное приложение для мониторинга физической активности: подсчёт шагов, дистанции, калорий. Интеграция с Google Fit/Apple Health, графики прогресса, push-уведомления о достижении целей

Вариант 45. Карточка книжной библиотеки с OCR-распознаванием

- **Предметная область:** Библиотечное дело
- **Технологии:** Python + Tesseract + PyQt5
- **Описание:** Десктопное приложение для каталогизации книг. Функция распознавания текста с титульного листа (OCR) для автоматического заполнения полей (автор, название, издательство). Поиск по каталогу, генерация QR-кодов для книг

Вариант 46. 3D-редактор для параметрического размещения объектов

- **Предметная область:** 3D-моделирование
- **Технологии:** Python + Blender API + NumPy

- **Описание:** Модуль-расширение для Blender, реализующий параметрическое размещение геометрических объектов в заданном пространстве (аналог Corona Scatter). Управление плотностью, масштабом, поворотом на основе карт текстур

Категория: Мультиагентные системы и ИИ

Вариант 47. Мультиагентная система для автоматизации техподдержки

- **Предметная область:** Сервисное обслуживание
- **Технологии:** Go + Python + LangChain
- **Описание:** Система агентов для обработки тикетов техподдержки. Агенты: классификатор (определяет категорию), анализатор (извлекает ключевые данные), генератор ответов (LLM), маршрутизатор (назначает ответственного). Оркестратор на Python управляет очередью [ЛР №13].

Вариант 48. Интеллектуальные агенты для игры в жанре "гонки на самолётах"

- **Предметная область:** Компьютерные игры, ИИ
- **Технологии:** C++ + Unreal Engine + Python
- **Описание:** Разработка агентов (ботов) для гоночной игры. Использование методов reinforcement learning для обучения управлению. Python для тренировки моделей, C++ для интеграции с игровым движком

Вариант 49. Распределённая система сбора и анализа данных с IoT-устройств

- **Предметная область:** Интернет вещей (IoT)
- **Технологии:** Rust + Go + MQTT
- **Описание:** Rust-агенты на устройствах (экономичные, безопасные), Go-шлюзы для сбора данных, Python-аналитик для обнаружения аномалий и прогнозирования состояния оборудования. Дашборд в реальном времени [ЛР №14].

Вариант 50. AI-ассистент для разработчика с интеграцией в IDE

- **Предметная область:** Инструменты разработчика
- **Технологии:** Python + FastAPI + LSP
- **Описание:** Сервис, предоставляющий подсказки по коду, генерацию тестов и документации через LLM. Интеграция с VS Code через Language Server Protocol (LSP). Локальный запуск моделей (Ollama) для конфиденциальности [ЛР №12].

4 ЭТАПЫ ВЫПОЛНЕНИЯ КУРСОВОГО ПРОЕКТА

Этап	Срок (ориентировочно)	Результат
1. Выбор темы и утверждение задания	1-2 неделя	Задание, подписанное руководителем
2. Анализ предметной области	3-4 неделя	Обзор аналогов, постановка задачи
3. Проектирование архитектуры	5-6 неделя	Диаграммы (UML, PlantUML), выбор технологий
4. Разработка ядра системы	7-9 неделя	Работающий прототип, репозиторий на GitHub
5. Реализация полной функциональности	10-12 неделя	Готовый программный продукт
6. Тестирование и отладка	13 неделя	Тесты, отчёт о покрытии, исправление багов
7. Документирование	14 неделя	Пояснительная записка, README, API docs
8. Предзащита	15 неделя	Презентация, демо
9. Защита проекта	Сессия	Финальная презентация, ответы на вопросы

4 ЭТАПЫ ВЫПОЛНЕНИЯ КУРСОВОЙ РАБОТЫ (ПРОЕКТА)

Этап	Недели	Содержание работ	Результат
1. Организационно-аналитический	1–2	Выбор темы, утверждение задания, сбор материалов, анализ предметной области и существующих аналогов, формулирование требований к программному продукту.	Задание на курсовую работу (проект), подписанное руководителем; аналитический обзор (раздел пояснительной записки).

2. Проектирование архитектуры	3–4	Разработка архитектуры приложения: выбор технологического стека, проектирование модулей, создание диаграмм (UML, PlantUML, C4), определение интерфейсов взаимодействия.	Архитектурная схема, диаграммы, описание в пояснительной записке.
3. Разработка ядра системы	5–7	Реализация ключевых модулей, создание прототипа, настройка репозитория (Git, Git flow), написание базовых тестов, первичная интеграция компонентов.	Рабочий прототип, репозиторий на GitHub, начальная версия кода.
4. Реализация полной функциональности	8–9	Завершение разработки всех запланированных функций, интеграция внешних сервисов (при необходимости), оптимизация производительности.	Готовый программный продукт с полной функциональностью.
5. Тестирование, отладка и безопасность	10–11	Модульное и интеграционное тестирование, статический анализ кода (SAST), проверка зависимостей (SCA), исправление выявленных ошибок, подготовка тестовых отчётов.	Отчёты о тестировании, покрытие кода, исправленные дефекты, обеспечение безопасности.
6. Документирование и подготовка к защите	12	Оформление пояснительной записки, создание презентации, подготовка демонстрационного стенда (если требуется), предзащита.	Полная пояснительная записка, презентация, готовность к защите.
7. Защита работы (проекта)	Зачётная неделя / сессия	Публичная защита курсовой работы (проекта), ответы на вопросы комиссии.	Оценка, рецензия на КР(КП)

ЗАКЛЮЧЕНИЕ

Курсовая работа (проект) по дисциплине «Методы и технологии программирования» является итоговой работой, демонстрирующей владение современными языками программирования (Python, C++, Go, Rust), инструментами и методологиями разработки. Успешное выполнение проекта

позволяет не только получить высокую оценку, но и сформировать портфолио из реального программного продукта, что является конкурентным преимуществом при трудоустройстве

Выполнение работы (проекта) предполагает использование всех знаний, полученных в ходе лабораторных работ №1-16, включая:

- Системы контроля версий и CI/CD [ЛР №1]
- Современные подходы к программированию [ЛР №2-4]
- Объектно-ориентированное и функциональное программирование [ЛР №3-4]
- Веб-разработку и базы данных [ЛР №6-7]
- Мультиязычную разработку (Go, Rust) [ЛР №9-10]
- Контейнеризацию [ЛР №11]
- AI-ассистированную разработку [ЛР №12]
- Мультиагентные системы [ЛР №13]
- Data Engineering [ЛР №14]
- Безопасность разработки [ЛР №15]

Рекомендуемая литература и ресурсы

1. Макконнелл С. Совершенный код. — Microsoft Press, 2023
2. Фаулер М. Архитектура корпоративных программных приложений. — Addison-Wesley, 2022
3. Документация Python 3.12: <https://docs.python.org/3/>
4. Документация Go: <https://go.dev/doc/>
5. Документация Rust: <https://www.rust-lang.org/learn>
6. Документация FastAPI: <https://fastapi.tiangolo.com/>
7. Docker Documentation: <https://docs.docker.com/>
8. GitHub Actions Documentation: <https://docs.github.com/actions>
9. Refactoring Guru — паттерны проектирования: <https://refactoring.guru/ru>
10. Курс по промпт-инжинирингу: <https://www.deeplearning.ai/courses/>